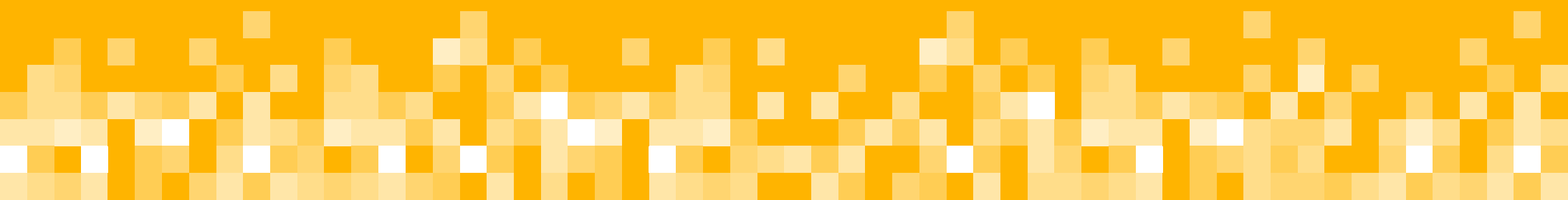


Moduł 3

Refaktoryzacja do wzorca Aggregate

Algorytmy działań refaktoryzacyjnych

Przywracanie enkapsulacji

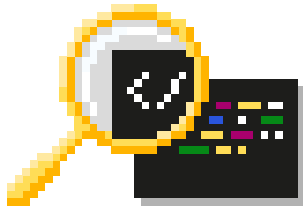


Zbudowanie bezpieczeństwa zmiany



Zbudowanie bezpieczeństwa zmiany

1.



Gdzie zmianę wprowadzić?

Wybierz miejsca, gdzie chcesz wprowadzać enkapsulację

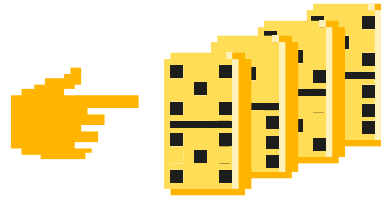
Na które miejsca się decydujesz? Gdzie zmiany nie wprowadzasz?

- jakie dane są niespójne w systemie?
- jakie są reguły i związane z nimi dane? Pomóc może Event Storming, analiza "ifów" w kodzie, rozmowa z ekspertami biznesowymi

Techniki

- biznesowe nazwanie intencji settera albo kombinacji setterów
- może w obiekcie są jakieś dane typowo odczytowe? W takim wypadku nie służą do reguł

2.



Określenie wpływu zmiany

Często warstwa serwisów, gdzie znajduje się implementacja niezmienników

Pułapki

- obiekt może być czytany przez refleksję

3.



Zapewnienie bezpieczeństwa

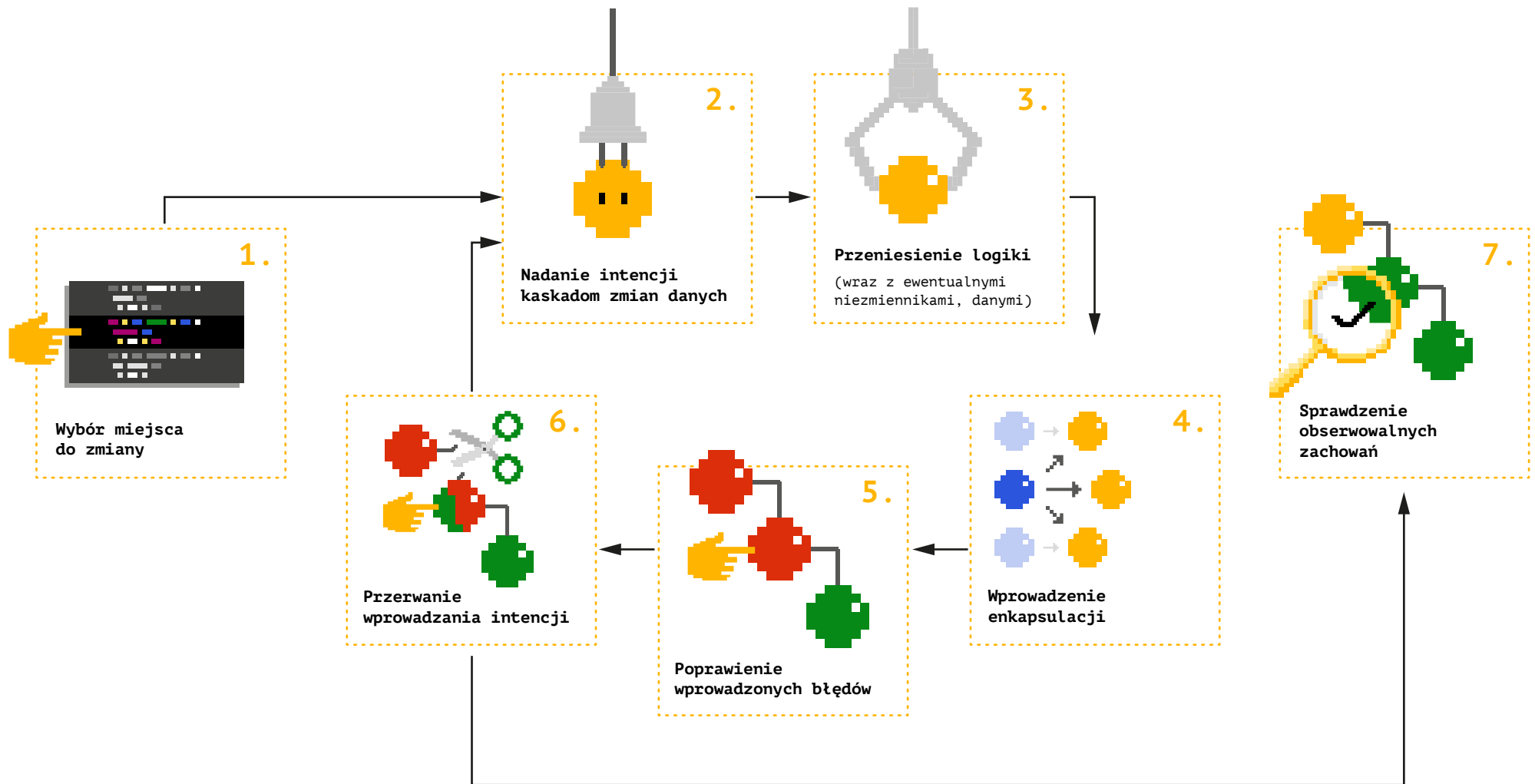
Test integracyjny jeśli w procedurze/serwisie jeśli sporo się dzieje i dokonujemy sporo zmian

brak testów jeśli przenosimy tylko settery do środka obiektu (test jednostkowy na obiekcie ewentualnie)

Techniki

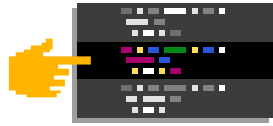
- test pokazuje intencje
- test testuje wpływ zmiany, a nie tylko samą zmianę danych (o ile może)

Wprowadzenie zmiany



Wprowadzenie zmiany

1.



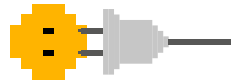
Wybór miejsca do zmiany

Wybierz miejsce od którego chcesz zacząć wprowadzenie enkapsulacji. Które dane są najważniejsze? Które trzeba zabezpieczyć?

Pytania pomocnicze

- jak trudna i jak ważna jest zmiana? (analiza ROI)
- które dane są niespójne?
- czy szybko dostanę informację zwrotną (fail-fast)?

2.



Nadanie intencji kaskadom zmian danych

Zastanów się nad kaskadą setterów - co one biznesowo oznaczają - nazwij to

Techniki

- nieużywane settery
- przekazanie przez konstruktor danych, które potem się nie zmieniają (użycie tylko z new()) - trochę jak move method + inline. Jeśli konstruktor jest duży to pomóc może wzorec builder
- zrozumienie, że sekwencja set() i get() to czasem chwilowe przypisanie danej tylko w celach odczytowych
- spojrzenie na sygnaturę metody serwisowej albo metody kontrolera może nam podpowieźć intencję

Pułapki

- podobne używane settery - to mogą być różne intencje
- settery używane raz z ifem, raz bez - to znowu mogą być różne intencje (np. Transit.setFrom)
- ustawianie obiektów w pewnym stanie na potrzeby testów może być trudne (wymaga setterów, konstruktorów albo metod biznesowych)
- kolejność setterów czasem może być istotna

3.



Przeniesienie logiki (wraz z ewentualnymi niezmiennikami, danymi)

Zacznij tworzyć metody biznesowe na obiekcie, przenosić tam zmianę danych, niezmienniki

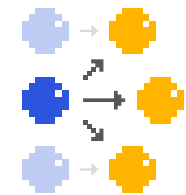
Pułapki

- kolejność setterów może mieć znaczenie
- do którego obiektu przenieść? Który obiekt powinien być korzeniem grafu obiektów?
- reguła może być rozpięta na wielu obiektach, być może każdy z nich ma już enkapsulację, ale w takim wypadku, korzeń grafu musi zarządzać kolekcją innych obiektów - wymaga to przeniesienia jednego obiektu (lub kolekcji) do środka grafu

Techniki

- rozróżnianie logiki aplikacyjnej od domenowej (różne rodzaje ifów)
- ify związane z walidacją (np. transit == null, driver == null) niekoniecznie muszą być niezmiennikami
- sztuczka z metodami canDo() i do()

4.



Wprowadzenie enkapsulacji poprzez usunięcie setterów i zabezpieczenie getterów

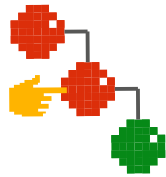
Zmiana stanu pól powinna odbyć się już tylko ze środka obiektu. Wyciąganie danych na potrzeby sprawdzenia niezmienników może odbyć się ze środka obiektu. Metody typu set nie są potrzebne, usuń je oraz zabezpiecz/usuń metody typu get

Techniki

- metody get mogą zwracać kopię
- część getterów może nie być potrzebna, np. na pobudki odczytowe
- usunięcie repozytorium (lub innego sposobu bezpośredniego dostępu) do ewentualnych obiektów podrzędnych, które teraz są w środku całego drzewa obiektów, modyfikowanego przez korzeń

Wprowadzenie zmiany

5.



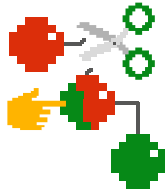
Poprawienie wprowadzonych błędów

Twoja zmiana spowoduje kaskadowe błędy w kolejnych miejscach systemu. Wiele miejsc mogło używać tych setterów

Pułapki

- bardzo dużo błędów, z których chcesz naprawić wszystko
- ustawianie obiektów w pewnym stanie na potrzeby testów może być problematyczne
- obsesyjna chęć pozbycia się każdego settera

6.



Przerwanie wprowadzania intencji

Nie naprawisz wszystkiego od razu. W bezpieczny sposób odetnij zmiany, które chcesz wykonać dopiero w przyszłości. Przywróć wcześniej usunięty setter/getter

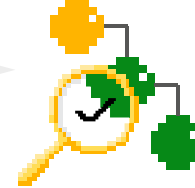
Pytania pomocnicze

- czy w tych kolejnych miejscach czuję bezpieczeństwo?
- czy muszę uzgadniać zmianę z innymi zespołami?
- czy wprowadzenie zmiany dotyczy miejsca dotkniętego problemem ze spójnością?

Techniki

- oznaczenie metody jako takiej, która nie powinna być już wywoływana w kodzie (komentarz/adnotacja)

7.



Sprawdzenie obserwowalnych zachowań

Czasem popełnisz błąd. Na szczęście system jest na to przygotowany. Wyłap błąd i go popraw za pomocą wcześniej przygotowanych mechanizmów bezpieczeństwa - np. testów automatycznych.

Usuń duplikację z testów. Większość testów można przenieść na poziom tworzonego obiektu.